

1 Aug 26

CSA1407-Compiler Design

Co2: Assignment tool 1

R. Balaji

192324255

AIDS

① Predictive parser

Sol:

(1111)

$E \rightarrow TE'$

$E' \rightarrow +TE' | \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' | \epsilon$, $F \rightarrow (E) | id$

$Follow(E) \rightarrow \{ \$,) \}$

$Follow(E') \rightarrow \{ \$,) \}$

$Follow(T) \rightarrow \{ +, \$,) \}$

$Follow(T') \rightarrow \{ +, *, \$,) \}$

$Follow(F) \rightarrow \{ +, *, \$,) \}$

Stack

\$E

\$E+

\$E+T

\$E+T(id

\$E+T

\$E+A

\$E+T

\$E+T(id

\$E+T(id

\$E+T(id

\$E+T(id

input

id + id * id \$

id + id * id \$

id + id * id \$

+ id * id \$

id * id \$

id * id \$

id * id \$

* id \$

* id \$

id \$

\$

Follow and FIRST

$FIRST(E) \rightarrow \{ (, id \}$

$FIRST(E') \rightarrow \{ +, \epsilon \}$

$FIRST(T) \rightarrow \{ (, id \}$

$FIRST(T') \rightarrow \{ *, \epsilon \}$

$FIRST(F) \rightarrow \{ (, id \}$

Action

shift. $E \rightarrow TE'$

shift $+ \rightarrow FT'$

shift $F \rightarrow id$

reduce $E' \rightarrow +TE'$

shift $E' \rightarrow +TE'$

shift/reduce $F \rightarrow +FT'$

shift $F \rightarrow id$

reduce $T' \rightarrow +E'T'$

shift $E' \rightarrow +E'T'$

reduce $T \rightarrow FT'$

reduce $T' \rightarrow \epsilon$

Accept $E' \rightarrow \epsilon$

② Shift reduce Parsing:

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow Pd$$

<u>Stack</u>	<u>input</u>	<u>Action</u>
\$	Pd + Pd * Pd \$	shift
\$Pd	+ Pd * Pd \$	reduce
\$E	+ Pd * Pd \$	shift
\$E +	Pd * Pd \$	reduce shift
\$E + Pd	* Pd \$	Reduce.
\$E + E	* Pd \$	reduce
\$E + E *	Pd \$	reduce
\$E + E Pd	Pd \$	shift
\$E + E * Pd	\$	shift Reduce
\$E + E * E	\$	Reduce
\$E + E	\$	Reduce
\$E	\$	Reduce
		Accepted.

⑤ SLR, LALR, CLR . ($S \rightarrow LR$, $L \rightarrow * R | Pd$, $R \rightarrow L$)

$$D_0: S' \rightarrow S; S \rightarrow * L = R; L \rightarrow * R | Pd; R \rightarrow L$$

$$D_1: S' \rightarrow S \checkmark$$

$$D_2: S \rightarrow L = R; R \rightarrow L$$

$$D_3: S \rightarrow R$$

$$D_4: L \rightarrow * (R; R \rightarrow * L; L \rightarrow * R | Pd$$

$$D_5: L \rightarrow Pd \checkmark$$

$$D_6: S \rightarrow L = R; R \rightarrow * L; L \rightarrow * R | Pd$$

$$D_7: L \rightarrow * R \checkmark$$

$$D_8: R \rightarrow C \cdot \checkmark$$

$$D_9: S \rightarrow L = R \cdot$$

$$(Pd = *Pd)$$

<u>Stack</u>	<u>input</u>	<u>Action</u>
	$Pd = *Pd \$$	shift
0	$= *Pd \$$	Reduce
0 Pd 5	$= *Pd \$$	shift
0 L 2	$= *Pd \$$	shift
0 L 2 = 6	$Pd \$$	shift
0 L 2 = 6 * 4	$Pd \$$	shift
0 L 2 = 6 + 4 Pd 5	$\$$	Reduce
0 L 2 = 6 + 4 R 8	$\$$	Reduce
0 L 2 = 6 R 9	$\$$	Reduce
0 L 2 = 6 R 9	$\$$	Reduce
0 S 1	$\$$	Accept

Q4 → PEG Evaluation

- ① Ambiguity Resolution : uses ordered choice (1). Tests
The first successful match commits immediately.
- ② Comparison :
 - Expressiveness : can parse non-context-free constructs ($a^n b^n c^n$)
 - Implementation : simple recursive descent (DR)
 - Backtracking : can cause severe performance drop without cache.
- ③ Python Adaptation : switched to PEG in 3.9 to eliminate
LL(1) limitations for complex syntax like structural pattern
- ④ Limitations : cannot handle left recursion directly > ordered choice.
- ⑤ Suitability : Best for modern programming languages. unsuitable
where full context free ambiguity checking.

5) Advanced (LL(1)) parser evaluation.

$S \rightarrow TS'$, $S' \rightarrow +TS' / \epsilon$, $T \rightarrow PT$, $T' \rightarrow *FT' / \epsilon$, $F \rightarrow (S) / Pd$

$FIRST(S) \rightarrow \{Pd, \epsilon\}$	$FOLLOW(S) \rightarrow \{\$, \epsilon\}$
$FIRST(T) \rightarrow \{Pd, \epsilon\}$	$FOLLOW(S') \rightarrow \{\$, \epsilon\}$
$FIRST(F) \rightarrow \{Pd, \epsilon\}$	$FOLLOW(T) \rightarrow \{+, \$, \epsilon\}$
$FIRST(S') \rightarrow \{+, \epsilon\}$	$FOLLOW(T') \rightarrow \{+, \$, \epsilon\}$
$FIRST(T') \rightarrow \{*, \epsilon\}$	$FOLLOW(F) \rightarrow \{*, +, \$, \epsilon\}$

$FIRST(\alpha) \cap FIRST(\beta) = \emptyset$ & $FIRST(S') \cap FOLLOW(S') = \emptyset$

Grammar is valid LL(1).

stack	input	Action
ϕS	$Pd + Pd * Pd \$$	$S \rightarrow T S'$
$\$ S' + F$	$Pd + Pd * Pd \$$	$F \rightarrow Pd$
$\$ S' T Pd$	$+ Pd * Pd \$$	$T' \rightarrow \epsilon$
$\epsilon S' T$	$+ Pd * Pd \$$	$S' \rightarrow + TS$
$\$ S'$	$+ Pd * Pd \$$	$+$
$\$ S' T'$	$+ Pd * Pd \$$	$T \rightarrow PT$
$\$ S' T' F$	$Pd * Pd \$$	$F \rightarrow Pd$
$\$ S' T' Pd$	$Pd * Pd \$$	Pd
$\$ S' T' Pd$	$* Pd \$$	$T' \rightarrow * P T'$
$\$ S' T' Pd$	$* Pd \$$	$*$
$\$ S' T' Pd$	$Pd \$$	$F \rightarrow Pd$
$\$ S' T' Pd$	$Pd \$$	Pd
$\$ S' T' Pd$	$\$$	$S' \rightarrow \epsilon$
$\$ S' T' Pd$	$\$$	Accept

⑥ Comparative Analysis of SLR VS LALR VS CLR

SLR(10 states): Suffer from shift reduce conflict in state 2
 (because FOLLOW(R) is too broad.
CLR(13 states): Resolves all conflicts by maintaining explicit 1-look
LALR(10 states): merges all identical items of CLR.
 Efficiency choice: LALR is most efficient, balances min state count.

⑦ operator precedence parsing:

operator	Precedence	precedence	precedence	precedence
\$	\$	+	*	\$
\$	\$	→	→	→
+	→	→	→	→
*	→	→	→	→
\$	→	→	→	→

Parse Tree:

Stack

input

Action

\$	\$d + d d d \$	shift
\$ d	+ d d d \$	Reduce
\$ E	+ d d d \$	shift
\$ E +	d d d \$	shift
\$ E + d	* d d \$	Reduce
\$ E + E *	* d d \$	shift
\$ E + E *	+ d d \$	shift
\$ E + E * d	* d d \$	Reduce
\$ E + E * E	d d \$	Reduce
\$ E + E	\$	Reduce
\$ E	\$	Accept.

⑧ Ambiguity removal and parsing strategy

unambiguous Grammar:

$$E \rightarrow E - T \mid T$$

$$T \rightarrow T / F \mid F$$

$$F \rightarrow Pd$$

<u>Stack</u>	<u>input</u>	<u>Action</u>
\$	Pd Pd Pd \$	shift
\$ Pd	- Pd / Pd \$	Reduce
\$ F	- Pd / Pd \$	Shift
\$ T	- Pd Pd \$	reduce
\$ E -	Pd / Pd \$	shift
\$ E - Pd	/ Pd \$	reduce
\$ E - F	/ Pd \$	reduce
\$ E - T	/ Pd \$	reduce
\$ E - T /	Pd \$	reduce shift
\$ E - T / Pd	\$	shift
\$ E - T / F	\$	reduce
\$ E - T	\$	reduce
\$ E	\$	reduce -
		Accept

⑨ PEG vs CFG parsing strategy Evaluation

$$E \leftarrow T \mid T' \mid E' \mid T$$

$$T \leftarrow F * T \mid F$$

$$F \leftarrow Pd$$

Ordered choice Behaviour

unlike a CFG, non-deterministic choices, PEG evaluates left-to-right (A/B). If A succeeds, B is never evaluated avoiding ambiguities.

Performance & Backtracking

Standard PEG with backtracking $O(2^N)$ worst case.

Pakrat PEG: $O(N)$ time complexity $O(N)$ space cost.

⑩ Design a Hybrid Parsing Strategy

System workflow!

[Input code] $\rightarrow$ [LL(1) Top-Down parser] [control flow, declarations]
 $\rightarrow$ [LR(1) Bottom up parser]

Design justification!

LL(1) handles simple structural constructs.

LR(1) handles operator expressions.

⑪ Unambiguous Grammar with custom operators.

Precedence rules!

- 1) $\wedge$ (Right-associative, highest)
- 2) $\%$ (Left-associative, middle)
- 3) $+$, $-$ (Left-associative, lowest)

Unambiguous grammar!

1) $E \rightarrow E + T \mid E - T \mid T$

2) $T \rightarrow T \wedge P \mid P$

3) $P \rightarrow P \cdot P \mid F$

4) $F \rightarrow Id$

⑫ Develop a New parsing Algorithm!

Python!

```
def parse_exp(rbp=0):
```

```
    token = advance()
```

```
    left = token.nud()
```

```
    while rbp < current.token.obp:
```

```
        token = advance()
```

```
        left = token.led(left)
```

```
    return left.
```

Performance

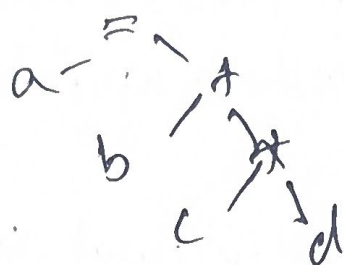
Time complexity = $O(N)$

Space complexity = $O(N)$

⑬ Compiler front end design ($a = b + c * d$)

① Lexical Analysis

Syntax:



Three Address Code generation

$$t_1 = c * d$$

$$t_2 = b + t_1$$

$$a = t_2$$

⑭ PEG-Based parsing System:

Convert CFG to PEG

$$E \leftarrow + \mid + E \mid \epsilon$$

$$T \leftarrow F * F \mid F$$

$$F \leftarrow P d$$

① evaluates $Pd + Pd * Pd$, $E \rightarrow T + E$

② F consumes the first Pd .

③ matched $+$, consumes E , ϵ consumes second Pd .

④ matched $*$ F consumes third Pd .

⑤ No backtracking occurs for valid inputs, parse tree is constructed.

Babji K
192324255